

Overall Lab Schedule

Experiments	Content	Week
1	Aircraft War System Analysis and Singleton Pattern	Week 11 Monday: 18:45-22:05 PM Room T2510 
2	Factory Pattern	Week 12 Thursday: 14:00-17:20 PM Room T2510 
3	Strategy Pattern	Week 14 Tuesday: 14:00-17:20 PM Room T2510 
 4	Audio Control and Multithreading	Week 16 Monday: 14:00-17:20 PM Room T2510 



Experiment 4: Audio Control and Multithreading

Objective

- Understand the lifecycle and basic concepts of threads in Java
- Master two thread implementation methods (Runnable interface, Thread class inheritance)
- Use multithreading to manage sound effects and switch between firepower supply states



Experiment Tasks

1. Sound Effect Requirements

Play appropriate sound effects when:

- a bullet hits an enemy, a power-up supply takes effect, or the game ends.
- **when the Boss enemy appears**, its dedicated BGM should start looping; this music must stop when the Boss is destroyed or the game ends.

2. Firepower Supply State Timing

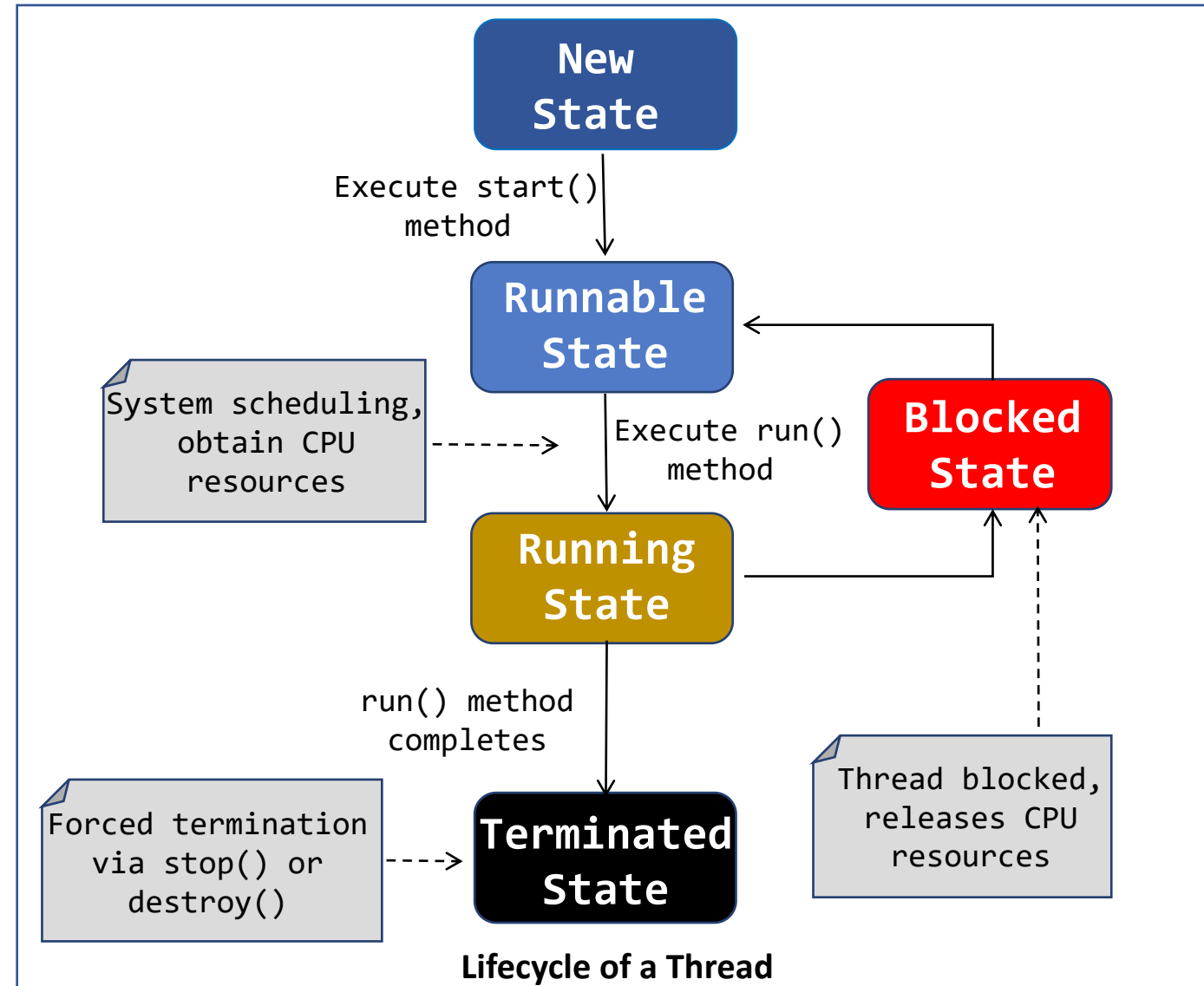
When a firepower supply is activated:

- the hero aircraft should switch to its enhanced firing trajectory and **maintain** this state for exactly **5 seconds**.
- then automatically **reverts** to the default straight-shot trajectory.

Experiment Steps: Multithreading in Java

Java provides built-in support for **multithreading**, allowing multiple threads of execution to run **concurrently** within a single program.

A thread is a dynamically executing process that also undergoes a **lifecycle**—from creation to termination.



Experiment Steps: Ways to Implement Multithreading

Two commonly used approaches:

- Implementing the Runnable interface
- Extending the Thread class itself to achieve multithreading.

(1) Implementing the Runnable interface

The Runnable interface can be implemented via a top-level class, static nested class, or anonymous inner class. As a **functional interface** with only one abstract run() method, it can also be instantiated concisely using a **lambda expression**.

Experiment Steps: Ways to Implement Multithreading

(1) Implementing the Runnable interface

```
public class RunnableTest {  
    public static void main(String[] args) {  
  
        Runnable r = () -> {  
            try {  
                for (int i = 0; i < 3; i++) {  
                    System.out.println("【" + Thread.currentThread().getName() + "】 " +  
                        "Thread is running, current loop count: " + i);  
  
                    Thread.sleep(2000);  
                }  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        };  
  
        // Create and start two threads that execute the same  
        new Thread(r, "Thread-1").start();  
        new Thread(r, "Thread-2").start();  
    }  
}
```

```
RunnableTest x  
C:\Users\duskphantom\.jdk\openjdk-24.0.2+12-54\bin\java.exe "-javaag  
【Thread-2】 Thread is running, current loop count: 0  
【Thread-1】 Thread is running, current loop count: 0  
【Thread-2】 Thread is running, current loop count: 1  
【Thread-1】 Thread is running, current loop count: 1  
【Thread-1】 Thread is running, current loop count: 2  
【Thread-2】 Thread is running, current loop count: 2  
  
Process finished with exit code 0
```

Experiment Steps: Ways to Implement Multithreading

(2) Extending the Thread class

Create a Thread subclass, override its run() method directly. Instantiate the subclass and invoke start() to start the thread.

```
public class MyThread extends Thread { 2 usages
    // Constructor to set the thread name
    public MyThread(String name) { 2 usages
        super(name);
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < 3; i++) {
                System.out.println("[" + Thread.currentThread().getName() + "] " +
                    "Thread is running, current loop count: " + i);
                Thread.sleep(millis: 5000); // Simulate work by sleeping for 2 seconds
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
public class MyThreadTest {

    public static void main(String[] args) {
        // Create and start two instances
        // of the custom thread class
        new MyThread(name: "Thread-1").start();
        new MyThread(name: "Thread-2").start();
    }
}
```


Experiment Steps: Multithreaded Implementation of Firepower Power-ups

➤ Tasks to be completed:

- When a firepower is activated, the hero aircraft must immediately switch to an enhanced firing trajectory and **remain in this state for exactly 5 seconds**.
- After the 5-second duration elapses, the aircraft should automatically **revert to its default straight-shot** trajectory.
- You must implement this timed behavior using multithreading **(via the Runnable interface)** to ensure it does not block the main game loop.

Experiment Steps: Multithreaded Implementation of Audio Control

This experiment provides a **MusicThread** class that extends **Thread** and overrides the `run()` method to control audio playback.

```
public class MusicThread extends Thread { 10 usages

    // Flag indicating whether the audio should loop continuously
    private boolean isRepeat = false; 3 usages

    // Flag to signal that playback should stop immediately
    private boolean isStop = false; 2 usages

    // Path to the audio file to be played
    private final String filename; 2 usages
```

```
/**
 * Constructor: initializes the music player
 * with a given file and repeat setting.
 */
public MusicThread(String filename, boolean isRepeat)
{
    this.filename=filename;
    this.isRepeat = isRepeat;
    reverseMusic();
}
```

It supports optional **looping** and **can be stopped** on demand.

Experiment Steps: Multithreaded Implementation of Audio Control

How to Use the MusicThread Class to Control Audio Playback?

```
/**
 * Main thread execution method.
 * Plays the audio repeatedly if isRepeat is true.
 */
@Override
public void run() {
    do {
        // Create a new input stream from the pre-loaded samples
        InputStream stream = new ByteArrayInputStream(samples);
        play(stream);
    } while (isRepeat);
}

/**
 * Signals the thread to stop playback immediately.
 */
public void stopPlay() { 3 usages
    this.isStop = true;
}
```

```
public class MusicTest {
    public static void main(String[] args) throws InterruptedException {
        // Create and start a looping background music thread.
        MusicThread bgm = new MusicThread( filename: "src/videos/bgm.wav", isRepeat: true);

        // Call start() to begin playing the audio in a separate thread
        bgm.start();

        // Let the background music play for 5 seconds.
        Thread.sleep( millis: 5000);

        //To stop the music (e.g., when the game ends )
        bgm.stopPlay();

        // Play a one-time sound effect (e.g., bullet hit).
        new MusicThread( filename: "src/videos/bullet_hit.wav", isRepeat: false).start();
    }
}
```

Experiment Steps: Multithreaded Implementation of Audio Control

To better manage audio playback, this experiment also provides a **MusicController class**. You are required **to complete the sections marked with TODO** in this class.

```
public class MusicController { no usages

    // File paths for audio resources
    public static String bgmMusic = "src/videos/bgm.wav"; 1 usage
    public static String bossMusic = "src/videos/bgm_boss.wav"; no usages

    // Keep references to looping music threads so we can stop them later
    private MusicThread bgmThread; 3 usages
    private MusicThread bossThread; no usages

    //Constructor: initializes the background music thread (set to loop).
    public MusicController() { bgmThread = new MusicThread(bgmMusic, true); }

    //Starts playing the background music (loops indefinitely).
    public void startBgmMusic() { bgmThread.start(); }

    //Stops the background music.
    public void stopBgmMusic() { bgmThread.stopPlay(); }
```

```
//Starts playing the boss music (should loop).
//TODO: Implement this method by creating and starting a looping MusicThread for bossMusic.
public void startBossMusic() { no usages
    // TODO
}

//Stops the boss music.
//TODO: Stop the bossThread if it is not null.
public void stopBossMusic() { no usages
    // TODO
}

//Plays the bullet hit sound effect (one-time playback).
//TODO: Create a new non-looping MusicThread for bulletHitMusic and start it immediately.
public void startBulletHitMusic() { no usages
    // TODO
}

//Plays the supply pickup sound effect (one-time playback).
//TODO: Create a new non-looping MusicThread for supplyMusic and start it immediately.
public void startSupplyMusic() { no usages
    // TODO
}

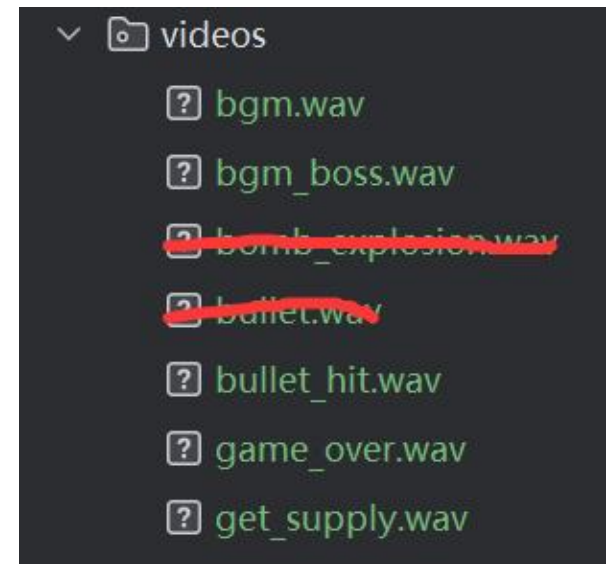
//Plays the game over sound effect (one-time playback).
//TODO: Create a new non-looping MusicThread for gameOverMusic and start it immediately.
public void startGameOverMusic() { no usages
    // TODO
}
```

Experiment Steps: Multithreaded Implementation of Audio Control

Once you have completed the implementation of the MusicController class, you can use it to refactor your game code and add sound effects.

Tasks to be completed:

- Loop **background music** during gameplay and stop it immediately when the game ends.
- Play sound effects on: **bullet hit**, **power-up activation**, and **game end**.
- Play **Boss music** (looping) as soon as the Boss appears.
- Stop Boss music when the Boss is destroyed or the game ends—whichever comes first.
- Use your **MusicController** class for all audio playback (thread-safe and non-blocking).



Assignment Submission

- **Submission requirements**

- ① Project code (the entire project folder)

- Compress the entire folder into a single archive file (.zip) for submission.

- **Deadline**

Submit your work to the **HITsz Grader** assignment submission platform within one week.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.



Let's begin the experiment!

THANK YOU